

Technology Intelligence Radar for Industry (TIRI)

Ranking scientific literature against industry use cases for high-recall technology monitoring.

Sylvain Fossier & Warren Fauvel

August 2026

Overview

About this Document

This whitepaper documents in detail the process and explorations of the Technology Intelligence Radar for Industry (TIRI) team as they tested, iterated and ultimately succeeded in developing a model for ranking scientific literature against industry Use Cases for high-recall technology monitoring.

[The technical repository for this project can be found here.](#)

The Project

An industrial R&D team decides where to put tens of millions of pounds across dozens of technology areas. To decide well, someone has to know what is happening within the science domain. The science is unreadable at volume: too much of it, written in specialist language, scattered across a dozen databases.

TIRI is a radar. An analyst writes down what they are looking for. A search returns a few hundred to a hundred thousand candidate papers. TIRI ranks that pile so the ones worth reading come first, and helps you decide if it is interesting or not, based on your own labelling.

Key Insights Summary

A model trained across multiple Use Cases performs worse than no model at all on a genuinely new one. Ensembles fitted on five Use Cases and tested on a sixth, unseen one caught only 8–20% of interesting papers, against 72–83% on the Use Cases they had been trained on — and a zero-label similarity ranker beat every one of them (0.693 against 0.610 ROC-AUC). Pooling does not merely fail to help; it actively hurts.

The reason is conceptual, not technical: relevance is a property of the (Use Case, paper) pair, not of the paper alone. Every feature the project first built described the paper in isolation — citation count, age, author count — and none of them read the Use Case, so no model trained on them could learn anything beyond which topic a paper belonged to. Ten features that explicitly compare paper to Use Case beat the full 384-dimension embedding on transfer to an unseen Use Case (0.642 against 0.537). Those ten are five keyword-match scores of the paper against the Use Case's five text fields — objective, problem statement, must-include terms, nice-to-have terms and domain — plus each score's percentile rank inside its own pool.

Use Case quality is a bigger lever than model scale. A Use Case inferred by a large language model from 30 labelled examples, rather than written by the analyst, improved the keyword-matching features by +0.065 and the model that reads the Use Case as prose by +0.057. By comparison, the entire performance spread across four open-weights model families from 20 billion to 397 billion parameters was 0.030.

Adversarial controls are cheap and worth building in from the start. Re-scoring each Use Case's papers against a deliberately wrong Use Case caught features that looked predictive but were encoding something else entirely. It also fired on one of the six Use Cases, and what it caught there was a defect in how that pool had been assembled rather than a fault in the features.

The last insight is the most obvious and the most painful: the dataset was too dense in interesting papers to optimise the chosen metric against. At 58% interesting, a policy of marking every paper interesting already scores $F2 = 0.872$ — above the untuned baseline, and only 0.020 below the tuned ensemble. That compressed the space available to optimise in, skewed model selection, and limited how much a more diverse set of advanced models could show.

A Note on Cutoff. TIRI as a project has progressed beyond the information documented here. The initial four-week explore, build and report cycle led to several insights that were easily started, but harder to fold into a concise report i.e. larger training sets, with lower prevalence of interesting and better Use Case construction.

1. Context: The Problem & Solution

Problem: Too much to read

A corporate R&D function commits large, multi-year budgets across many "solution spaces" at once. Beneath it sits a technology intelligence team, usually fragmented, serving several Use Cases per solution space. Beneath them sits the person who actually does the work, a technology analyst with deep knowledge of one specialism and a Use Case that changes every few months.

That analyst's problem has three parts:

- Volume. More science is published each year than any team can read.
- Language. It is written by specialists, for specialists, in vocabulary that shifts between subfields.
- Fragmentation. Search, triage, note-taking and reporting live in different tools, and nothing carries the analyst's judgement forward.

Example: Carbon Capture M&A

- **Business Strategy:** Acquire promising carbon-capture IP from university spin-outs.
- **Use Case:** low-energy industrial point solutions, amine-based reagents.
- **Terminology:** industry emissions, carbon capture, amines, reagents, efficient heat transfer systems, membranes...

To stay on top of the science for this Use Case and support the Business Strategy, the Analyst needs to:

- Formulate a good query that folds in specialist terminology OR set a broad search term and filter results more aggressively

- Run this query on several different sources, with varying query formats and functionality e.g. semantic search
- Skim read hundreds of results (in various tabs) to ensure accuracy, with no idea if things have been missed or not
- Start to read the most interesting of the results, often across multiple tabs, sometimes missing things due to interruptions and changes of layouts

Across hundreds of results perhaps a third are interesting. The analyst has time to read 10 per week. Which ten? That is the entire question TIRI answers.

The Solution Concept: A Radar for Science

The initial hypothesis was that given a clearly described "Use Case" and a labelled set of papers (interesting / not interesting), categorical, ordinal and natural-language features could be combined into a scoring mechanism fine-tuned to one analyst's needs. The workflow was defined as:

1. Analyst defines a Use Case to a given structure
2. Queries are built and retrieved from multiple data sources
3. Results are deduplicated and embedded
4. Analyst is presented with a normalised set of results in a clear interface
5. Analyst labels N papers interesting/not interesting
6. A pre-built, generalised model, is fitted to the new case

Issues with this approach

This project surfaced several challenges with that approach, which led to an improved workflow

- Use cases are hard to create from scratch, they needed structure
- Papers without abstracts are hard to label, this is generally an issue of paid/open source data
- In a new area for the analyst every paper is interesting and "interesting" can drift over time as they learn new things and refine their thinking

- Labels and Use Cases can silently be in conflict over time, as new knowledge emerges, or merely from mistakes
- Different labellers have different thresholds of interesting, meaning that one paper can be both interesting and not interesting for the same Use Case
- How many labels are enough to train a model? Labelling is tedious and repetitive, users have a threshold of about 25 papers p/sitting (c.30-45 mins)

An Improved Workflow

The optimal approach operates as a loop. Each pass through it sharpens the definition of "interesting" and interrogates the other. This results in better "raw material" for the model to work with.

Stage	What happens	Volume
Define the Use Case	The analyst fills in a structured Use Case (a JSON schema, LLM-assisted)	1 Use Case
Search / retrieve	Queries fan out across academic databases ¹ , results deduplicated	100–500 papers
Label	The analyst triages each paper: interesting / not interesting / pass	100–500 labels
Create / train model	Cleaning, exploratory analysis, feature engineering, ensemble, this is TIRI	—
Expand the harvest	The trained model labels a much larger pool	1k–100k papers
Refine the Use Case	The analyst re-labels where the model erred; the Use Case is revised; return to 1	—

¹ The data sources are OpenAlex, Crossref, arXiv, Semantic Scholar, PubMed, Europe PMC and CORE.

Additional Requirements

Because the setting is high-stakes, high-infosec industrial R&D, where corporate espionage and data leaks are a real risk, three further constraints applied from the outset:

- **Deterministic** - stochastic solutions such as large language models, though capable, are hard to trust if they return different results on every pass. The solution had to behave consistently
- **Auditable** - \$mm decisions require audit trails and provenance, linked to determinism, but providing the analyst with “cover” should they make a recommendation
- **Sovereign** - ideally, the solution should run in a Virtual Private Cloud, or even locally, requiring only open-source licences to run and sharing minimal data

2. Defining Success

Interesting is Subjective

It is overly simplistic but also true to say that the task is to define “interesting” mathematically. “Interesting” is subjective, both as a term and in the context of a Use Case. This report defines it as:

Interesting: Research containing relevant information to progress the solution to a specific Use Case.

This could be interpreted in several ways including:

- Information on a proven solution focused on the Use Case
- Information on an unproven solution focused on the Use Case
- Information on a solution that failed
- Information on an indirect solution that supports a direct solution

Ultimately the solution was multiple passes. Writing a Use Case, labelling data, training a model — each pass gives the analyst another opportunity to define their own version of “interesting” for that Use Case.

Ranking & Cutoff

An interesting tension exists due to the scope of this project and is part in the larger exploration of Industry R&D. Namely:

- Analysts have limited time - thus Ranking things by “interesting” is important, lending itself to metrics such as ROC and WSS
- Analysts do not want to miss things - thus “Interesting” requires a higher recall cut-off, lending itself to metrics such as $F2 @ t^*$ (where t is tunable boundary)

This report switches between both positions depending on what is being optimised: either helping the analyst read quickly, or making sure the interesting set is more complete. That is not ideal for model building and may be confusing to read, but it is the reality of the requirement as it stands. Wherever a number appears, the metric it is measured in is named.

Handling Errors

No model is without error, irreducible noise or optimisable, limited by data set or other technical constraints. Ranking systems make two kinds of mistakes, which cost different amounts.

- **A false positive**, the model flags a paper that turns out not to be interesting. Cost: the analyst wastes a few minutes reading it. Annoying, recoverable.
- **A false negative**, the model buries a paper that mattered. Cost: the insight is simply never found. Nobody knows it was missed. Unrecoverable.

Two standard measures capture this:

- Precision, of the papers the model flagged, what share were actually interesting?
"Only the right answers, but some were missed."

- Recall, of the papers that were actually interesting, what share did the model flag?
"All the possible right answers, but some wrong ones came along."

Key Metric: F2, double the Recall, single Precision

Precision and Recall cannot both be maximised. F-scores combine them into one number with an adjustable weight. TIRI uses F2, which counts recall as twice as important as precision. In plain terms:

"Don't miss weak signals. A weak signal that reaches the analyst can be discarded in thirty seconds. A weak signal that never reaches them is gone."

Of course precision is also important to reduce overwhelming analysts with information. For TIRI, the precision optimization is a downstream problem, reducing from 1000 potential papers to 50, where even half are not interesting, is a much more tractable problem for precision.

Limitations of F2

F2 depends on **how common interesting papers are** in the pool being scored. Across the six Use Cases, between 26% and 77% of decided papers are interesting, 58% pooled. In a real production harvest the figure is in the low single digits: the 28-project external corpus described in Future Work runs at 1.86% interesting overall.

That difference matters enormously, because a model that simply marks *everything* interesting scores a perfect recall of 1.0 and a precision equal to whatever share of the pile was interesting anyway. F2 for that do-nothing policy is therefore fixed by the pool alone:

Share of the pool that is interesting	F2 for marking everything interesting
26% — the sparsest Use Case (Soil microbiome)	0.64

Share of the pool that is interesting	F2 for marking everything interesting
58% — the six Use Cases pooled	0.87
77% — the densest Use Case (Solar cells for satellites)	0.94
1.86% — the external corpus, and closer to production	0.09

In a pool that is 77% interesting there is almost nothing to skip, so F2 cannot tell a good ranker from a lazy one. This is not a hypothetical: the do-nothing score on the pooled dataset is 0.87, and Section 6 shows the trained models landing either side of it. Precision, Recall and ROC-AUC² were therefore examined alongside F2 throughout.

² Receiver Operating Characteristic, area under the curve (ROC-AUC) is a performance metric for how well a model ranks items. In simple terms it is the probability that, picking one interesting paper and one not-interesting paper at random, the model puts the interesting one higher. A score of 0.5 is a model that is essentially guessing (a coin flip); 1.0 is a perfect ranking. Unlike F2 it does not move when the pool gets denser or sparser, which is why it is used throughout this report alongside F2.

3. The Data

Six Diverse Use Cases

The initial dataset was hand-labelled from the TIRI team's own working experience, augmented by desk research. Use Cases were hand-written to a defined structure (Title, Problem, Industry...), though this quickly proved hard to do consistently without training. Papers were labelled inside a simple application — described in-house as “tinder for academic research” — to speed the process up.

Use Case	Processed Papers
Low-carbon cement binders	690
Soil microbiome quality	602
Named entity recognition	541
Solar cells for LEO satellites	427
Carbon capture	323
Predicting technology developments	290
Total	2,873

Each row carries roughly 50 columns: bibliographic metadata (DOI, title, abstract, venue, citation count, year, authors), which database it came from, the analyst's label, and a 384-number “embedding”. A model that reads the unstructured text (title and abstract) and turns them into a list of numbers, positioned so that papers about similar things end up close together.

Data Volume

The initial harvest processed 2,873 papers across four target categories. *Pass* and *never triaged* look similar but carry an important distinction: *Pass* means an analyst looked and could not decide. *Never triaged* means no human ever saw it.

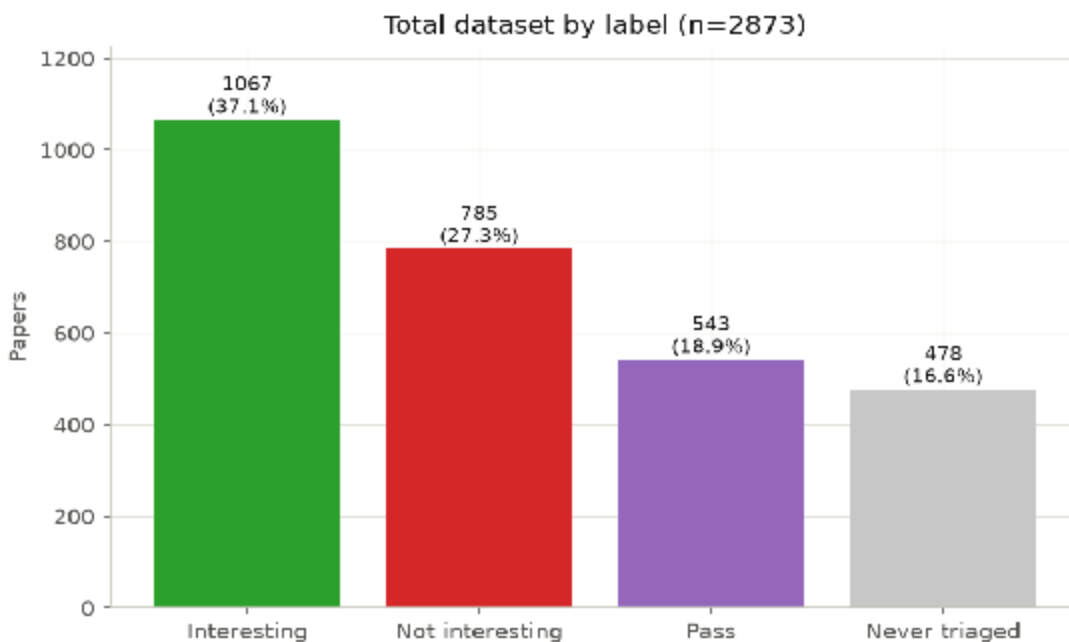


Figure 1. Every harvested paper, split by what the analyst did with it. Interesting is 1,067 papers — 37.1% of all 2,873 rows, but that denominator includes the 1,021 rows nobody ever decided on. Among papers that carry an actual decision the interesting rate is 58%, and 58% is the figure every model score in this report is measured against. Had the searches returned nothing worth reading, the green bar would sit near zero.

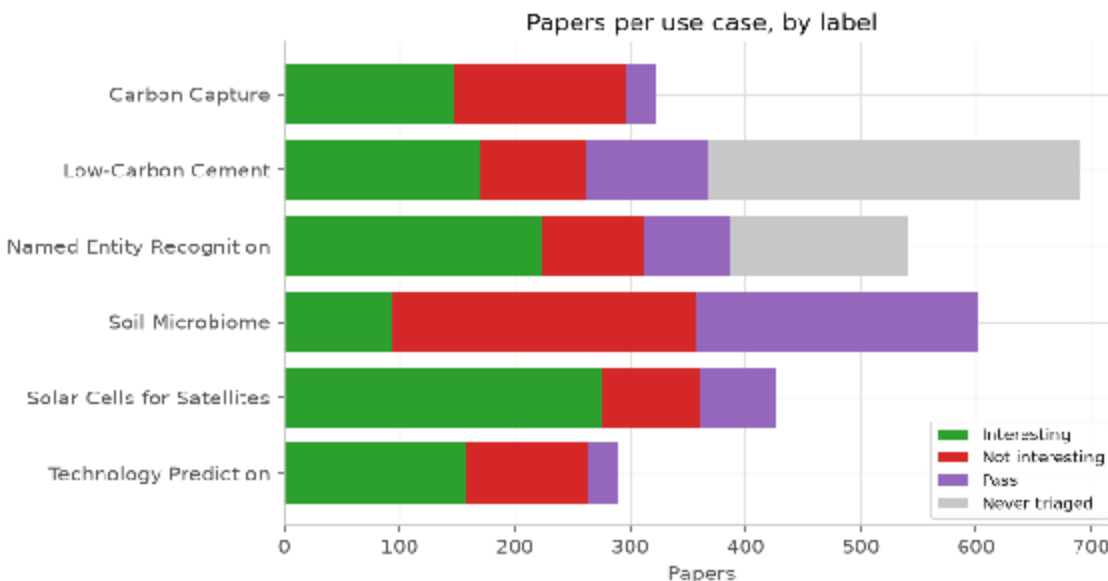


Figure 2. The same split, one bar per Use Case. Bar length is the paper count from the table above; what differs between Use Cases is how much of each bar is green. Low-Carbon Cement is largely grey because labelling stopped before its pool was exhausted, not because those papers were rejected.

Prevalence: the number that constrains every score in this report

Across the six Use Cases, 58% of decided papers are interesting — between 26% and 77% depending on the Use Case. A production harvest looks nothing like that. The larger external corpus described in Future Work runs at 1.86% interesting, roughly thirty times lower. Every metric in this report was therefore measured on a pool about thirty times denser in interesting papers than the pool the system would actually meet, and that is the single largest caveat on the numbers that follow. It is addressed in Future Work.

Missing Data

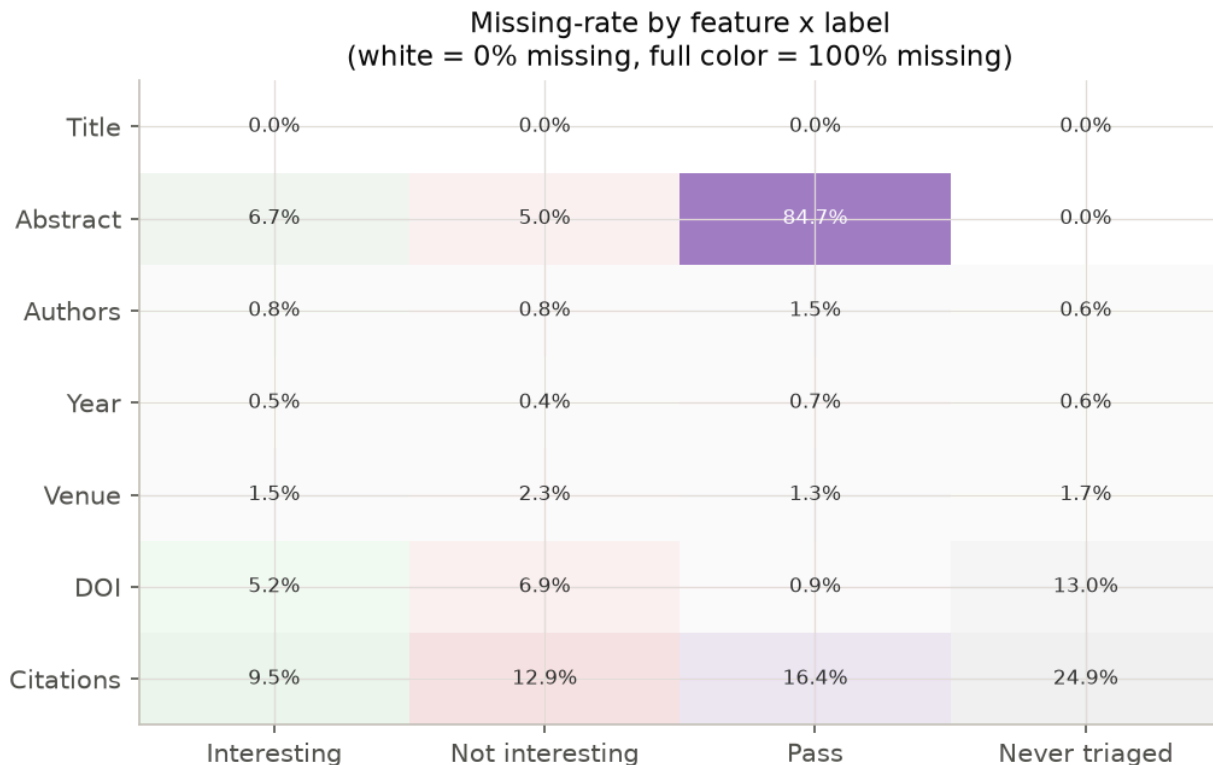


Figure 3. Missing-data rate for each field, split by label. If a field's completeness had nothing to do with the analyst's decision, that row would be flat across all four columns. The abstract row is the finding: it is not flat.

NULL is not 0

"Someone looked and there was nothing" and "nobody ever found out" are different facts. A missing citation count means no database recorded one, not that the paper has zero citations. Every feature built over a sparse field emits a genuine *missing* marker plus a flag, rather than quietly filling in a zero and inventing data.

Pass was highly correlated with missing abstracts, an artefact of the harvest being blocked by some publishers. 84.7% of pass papers have no abstract, against 5–7% for

decided papers. A paper with no abstract cannot be labelled, so *Pass* and *Never triaged* were both dropped rather than treated as a third class.

Cleaning therefore produces two counts, and both appear in this report because they mark two different stages. Anywhere a score is quoted, it was measured on the 1,848.

Stage	Papers	What was removed at this step
Harvested	2,873	—
Carrying a usable interesting / not-interesting label	1,852	543 <i>pass</i> and 478 <i>never triaged</i>
The modelling table, after de-duplication	1,848	4 rows that were the same paper recorded twice

Other Data Features

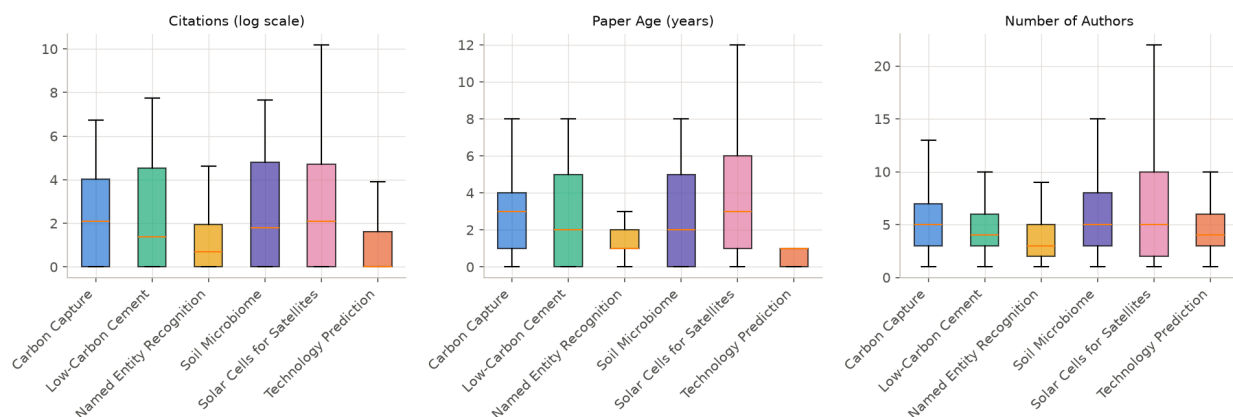


Figure 4. Citation count, paper age and author count, per Use Case. Median and mean are shown together on purpose: where they diverge badly the field is skewed, and a mean is the wrong summary. Solar cells for satellites has a median of 7 citations and a mean of 222 — one field, two completely different stories depending on which number is quoted.

- Citations are heavily skewed everywhere, the most-cited paper in the corpus has 26,506 citations, verified genuine.
- Age splits by field: the digital-technology Use Cases skew much younger. Technology prediction is almost entirely papers under one year old; solar cells for satellites reaches back to 1974.
- Author counts are higher in the hard sciences than in the computational ones.

Embeddings Analysis

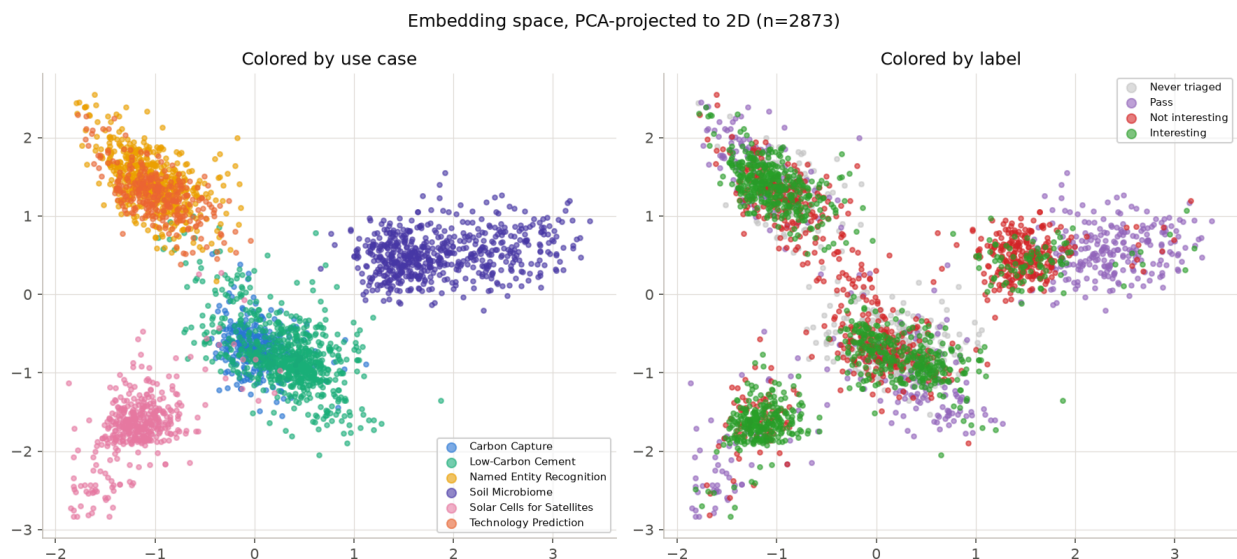


Figure 5. The same 2,873 papers plotted twice. Both panels compress each paper from its 384 numbers down to two, so it can be drawn on a page; papers that sit close together are papers the embedding considers similar. The panels differ only in what the colour means. Left, colour is the Use Case. Right, colour is the analyst's label.

- Left panel: six clean, well-separated clusters. The numbers know exactly which Use Case each paper belongs to.
- Right panel: green and red thoroughly mixed inside every cluster. The numbers do not know which papers are interesting.

To put a figure on the left panel, a classifier was trained to predict *which Use Case a paper belongs to* from its embedding alone: **96.2% accuracy**, against 19% for always guessing the most common Use Case. So the embedding is not noisy. It is an extremely precise fingerprint of the topic — and topic is not the same thing as interesting.

That gap is the project.

4. Key Insight: A Single Model Cannot Fit Every Use Case

A Flawed Hypothesis

The obvious approach: pool all six Use Cases, train one model, ship it. On a random hold-out drawn from those same six Use Cases it scores well. But the product's real question is different — how will it do on a customer's brand-new Use Case?

To simulate that, one Use Case was hidden entirely, a model trained on the other five, and tested on the hidden one — rotating through all six so no single Use Case could carry the result. The results are below. In short: it does not transfer at all. Recall, the component that drives F2, suffers hardest. On Use Cases it has seen, a model catches 72–83% of interesting papers. On a new Use Case it catches 8–20%. Four out of five interesting papers would be lost.

Worse still, with zero labels, simply ranking papers by how similar they are to the Use Case scores 0.693 on a new Use Case — beating every supervised model trained on the other five (0.610).

A model trained only on other customers' Use Cases is worse than no model at all.

Model	On Use Cases it has seen, ROC-AUC*	On a brand-new Use Case, ROC-AUC*
Gradient boosting	0.816	0.505
Random forest	0.808	0.502
Logistic regression	0.759	0.536

Model	On Use Cases it has seen, ROC-AUC*	On a brand-new Use Case, ROC-AUC*
A coin flip	—	0.500

* Interpreting ROC-AUC: pick one interesting paper and one not-interesting paper at random, how often does the model rank the interesting one higher? 0.5 is a coin flip. 1.0 is perfect.

Testing the Errors

The reflex diagnosis is *overfitting*: too many features, model memorised the training data. Measurement says otherwise — 60–65% of the collapse is domain shift, not overfitting.

What was measured	Result	What it means
Predict <i>which Use Case</i> a paper belongs to, from its embedding alone	96.2% <i>(19% for always guessing the most common Use Case)</i>	The features encode topic identity, and topic identity is useless on an unseen topic
Subtract each Use Case's average position, to remove the topic	0.532 → 0.529	No effect. You cannot subtract your way out , the interesting direction is different in every Use Case
Ten features that compare paper to <i>Use Case</i> , against the full 384-number embedding	0.642 vs 0.537	Ten beat 384. It was never the feature count; it was the feature <i>kind</i>

Use Case and Papers

The “train one model” hypothesis sounds intuitive until you state the actual insight plainly.

Relevance is a property of the (Use Case, paper) pair, not of the paper. There is no such thing as an interesting paper. There is only a paper that is interesting for this Use Case.

Every feature the project had built up to that point described the paper alone: how often it was cited, how many authors, how new, what it was about. None of them read the Use Case. So the model had no way to learn a relationship, only a topic.

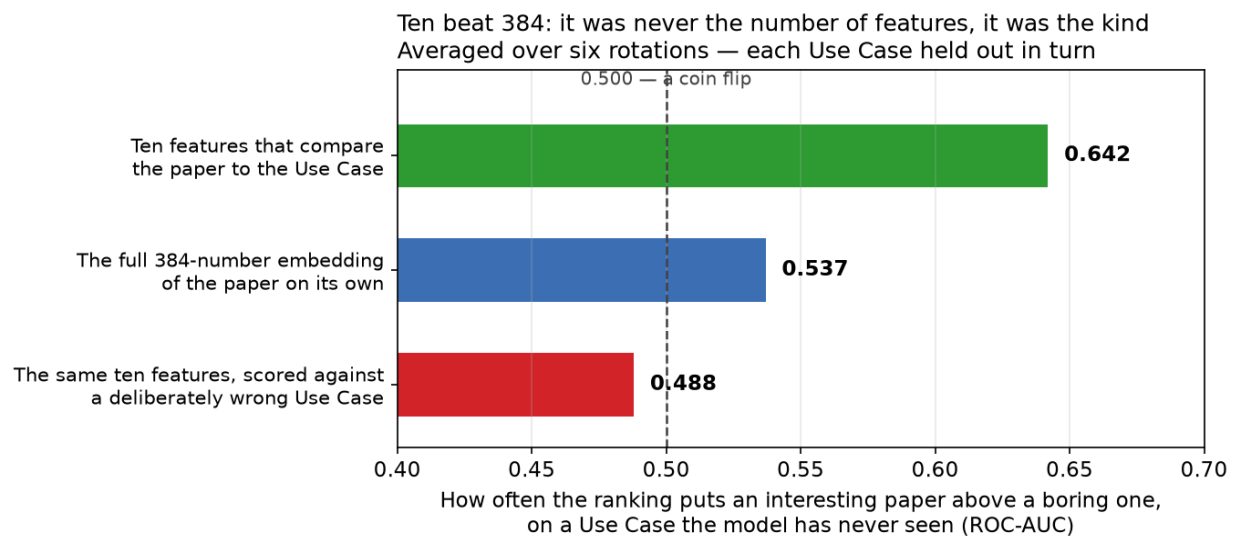


Figure 6. The same evaluation for three different feature sets, each held out on a Use Case it never saw, averaged over all six rotations. 0.500 is a coin flip. The 384-number embedding barely clears it; ten features that read the Use Case reach 0.642; and the same ten scored against the wrong Use Case fall below chance, which is the control described in the next subsection.

The ten features

This is the mechanism the rest of the report rests on, so it is worth stating exactly what the ten are. A structured Use Case has five text fields. Each paper's title and abstract is scored against each of those five fields with **Okapi BM25** — a standard keyword-match score from search engines, which rewards a paper for containing the Use Case's rarer, more distinctive words rather than its common ones. That gives five numbers. Each is then converted into a **percentile rank within its own Use Case's pool** — "this paper matched the objective better than 87% of the papers retrieved for this Use Case" — which gives five more.

Use Case field the paper is scored against	Raw match score	Rank of that score within the pool
Objective — the one-line statement of what the analyst is trying to achieve	1. bm25_obj	6. rank_bm25_obj
Problem statement — the longer prose description of the difficulty being solved	2. bm25_prob	7. rank_bm25_prob
Must-include terms — the analyst's curated list of non-negotiable terminology	3. bm25_must	8. rank_bm25_must
Nice-to-have terms — terminology that is a positive signal but not required	4. bm25_nice	9. rank_bm25_nice
Domain block — industry, application and technology focus, combined	5. bm25_dom	10. rank_bm25_dom

The ranks earn their place because a raw BM25 score rises with how long the paper is and how large the pool is, whereas a percentile rank does not — so a rank means the same thing in every Use Case, and a model fitted in one silo is reading a comparable

quantity in another. Nothing in the block is fitted on labels, which is why all ten can be computed for a brand-new Use Case before a single paper has been labelled.

Two details from the ablation are worth carrying forward. **BM25 alone accounts for essentially the whole effect** — 0.637 of the 0.642 — so the shipped block is these ten columns rather than the twenty-two originally built. And the win is not a length artefact: features that measure paper length alone reach 0.550, and removing them from the block costs 0.004.

Systematic Testing

A feature set that claims to "read the Use Case" might in fact be measuring something generic — abstract length, say, or how technical the writing is — and scoring well for the wrong reason. So every Use Case's papers were re-scored against a *deliberately wrong* Use Case and measured again. If the score survived, the features were not reading the Use Case and the block was thrown away.

Averaged over all six Use Cases	ROC-AUC on the held-out Use Case
Scored against its real Use Case	0.642
Scored against a deliberately wrong Use Case	0.488
<i>A coin flip</i>	<i>0.500</i>

Real Use Cases beat wrong ones on **5 of 6**, with a mean gap of **+0.155**. The wrong-Use-Case version lands slightly *below* a coin flip, which is the strongest form of pass this test can give: there is no generic residual left over once the correct Use Case is taken away.

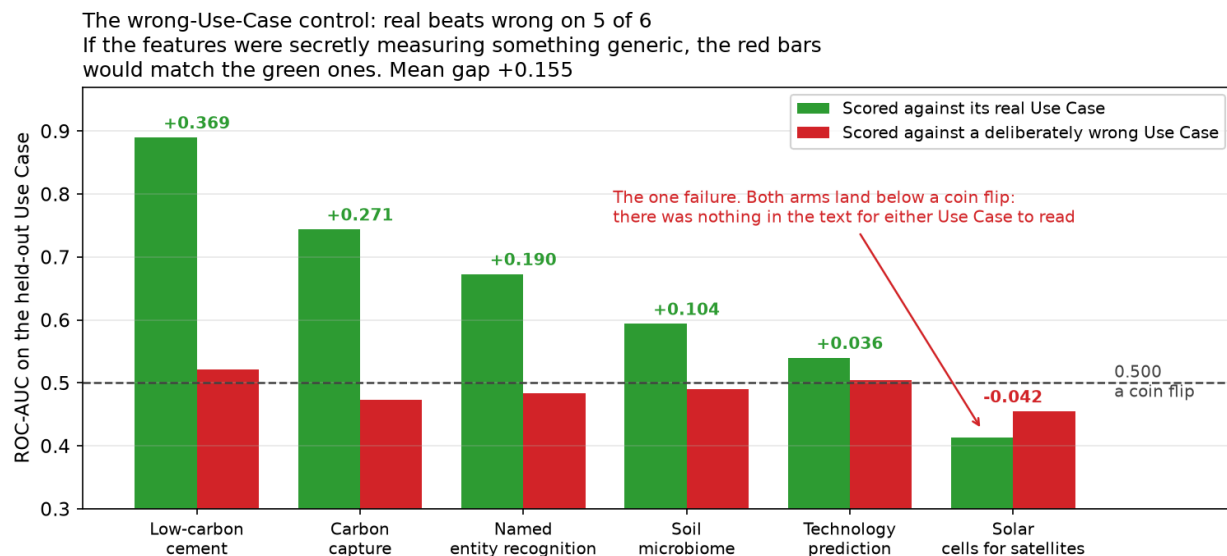


Figure 7. The control, Use Case by Use Case, ordered by the size of the gap. If the ten features were secretly measuring something generic, each red bar would sit level with its green one. Five pairs show a clear gap; one does not.

The one that failed: Solar cells for satellites

The sixth Use Case, **Solar cells for satellites**, failed the control: the real Use Case scored **0.413** and the deliberately wrong one **0.455**, a gap of **-0.042** in the wrong direction. The important detail is that *both* numbers sit below 0.500. This is not a case of the features scoring well for a bad reason — they scored worse than a coin flip either way. The control was reporting that there was nothing in the paper text for any Use Case, right or wrong, to read.

The reason is in how that pool was assembled, and it is a labelling-design fault rather than a modelling one. The analyst's own note records that the corpus was seeded from the reference lists of the field's review papers, so the starting pile is established canon — well-cited, foundational work. The Use Case then asked for approaches that improve *beyond what incumbent technologies currently deliver*. Seed a pool with canon and then reward going beyond canon, and the older canonical papers are "not interesting" by construction.

The consequence is measurable. The dividing line in that Use Case is publication vintage, not content: the interesting rate runs 33% for papers before 2018, 63% for 2018–20, and **98%** for 2020–22. Publication year on its own ranks that Use Case at **0.766**, well ahead of the 384-number embedding's 0.648. Topically near-identical papers sit on opposite sides of the boundary — one paper on perovskite film morphology from 2015 is not interesting, a closely related one on film grain size from 2016 is interesting.

Two things follow, and both became standing practice:

- **The control did its job.** A test that only ever passes is not a test. This is the case where it fired, and what it caught was a defect in the data rather than in the features.
- **This Use Case is never cited as evidence that the system finds interesting work.** A model can reach 0.805 on it using publication year and author count alone, which means it has learned this corpus's vintage distribution and should not be expected to transfer anywhere.

Three operating rules came out of this section:

1. Features must read the Use Case.
2. Models are fitted per Use Case, never pooled.
3. Below about 25 labels, ship the zero-label ranker — rank by similarity to the Use Case and use the top of that list to choose what to label next. Above 25, train.

5. Key Insight: Comparing Embedding Models

About Embeddings

The models never read words. An embedding model turns a paper's title and abstract into a long list of numbers, and everything downstream sees only that list. If two different papers come out with near-identical lists, nothing further down can tell them apart — the difference is already gone. So before tuning any ranker, the question was whether the free model the corpus arrived with was throwing signal away. Twelve candidates, one identical test each.

Each model was scored on a Use Case it had never seen, using PR-AUC (precision-recall area under the curve): one number for how well a ranking pushes the interesting papers to the top.

Averaged over all six Use Cases	Scored	Random shuffling scores	Real gain
Free local model	0.585	0.581	+0.004
Best paid model	0.632	0.581	+0.051

** PR-AUC does not start at 0.50 the way a coin flip does. Shuffling papers at random scores whatever fraction of the pile is interesting anyway, and these piles were deliberately dense — 26 to 77 interesting in every 100. That is why the "random shuffling" column is shown beside the score: the gain, not the score, is the result.*

The free model sits at chance: on an unfamiliar Use Case it has learned nothing. The paid model's gain is real, but a third of what a single Use Case had implied, and it clearly beats chance on only three of the six.

Where the paid models earn their money

Piles that are 26–77% interesting leave a ranking little to do. Real screening looks nothing like that: only 1.7 to 15 papers in 100 are interesting. On a published review of 1,970 papers containing 33 interesting ones, the ranking lets a researcher skip half the pile unread and still find 31 of the 33. That is the operating regime TIRI is built for.

6. Building the Model

Both models below are fitted on the same 1,848 rows, the same split and the same feature space — 33 non-embedding features, which include the ten Use-Case-relative scores of Section 4, plus the embedding block compressed to 50 dimensions — so any difference between rows is a difference of algorithm, not of data. **Every score in this section is F2**, the key metric of Section 2, where recall counts twice as much as precision. The split is grouped by first author, so the same author never appears on both sides of it.

Read the gap between the three columns as well as the height. A model that scores far better on data it trained on than on data it has not seen has memorised rather than learned; flatter is better, and *Test* is the column that matters because those rows were touched exactly once.

Baseline Model (logistic regression)

The baseline is a plain, untuned logistic regression over that feature space — deliberately simple, so that anything more elaborate has something honest to beat.

	Train (60%)	Validate (20%)	Test (20%)
Baseline , logistic regression, untuned, F2	0.841	0.820	0.790

Advanced Model (CatBoost + Random Forest + Logistic Regression)

Why an ensemble? Three different algorithms make three different *kinds* of mistakes. A gradient-boosting model finds sharp conditional rules; a linear model finds broad smooth trends; a random forest sits between them. Averaging their opinions cancels errors that

any one of them makes alone — three radars on different frequencies, cross-checking each other's contacts.

	Train (60%)	Validate (20%)	Test (20%)
Baseline , logistic regression, untuned, F2	0.841	0.820	0.790
Ensemble , CatBoost + random forest + logistic regression, F2	0.958	0.906	0.892

What these scores have to beat

An earlier draft of this report claimed an estimated practical ceiling of around 86% for this task. **That figure is withdrawn, because nothing in the project measures it.** A ceiling on this task would have to come from how often two competent people disagree about the same paper against the same Use Case, and that number does not exist here: the corpus was labelled by a single analyst, so there is no inter-annotator agreement to quote and no repeated-labelling study to draw on. Measuring it is the second item in Next Steps.

What *can* be measured is the opposite bound — the floor. The 370 unseen test papers are 58% interesting, so a policy of marking **every** paper interesting scores a perfect recall of 1.0, a precision of 0.576, and therefore **F2 = 0.872** without reading anything at all.

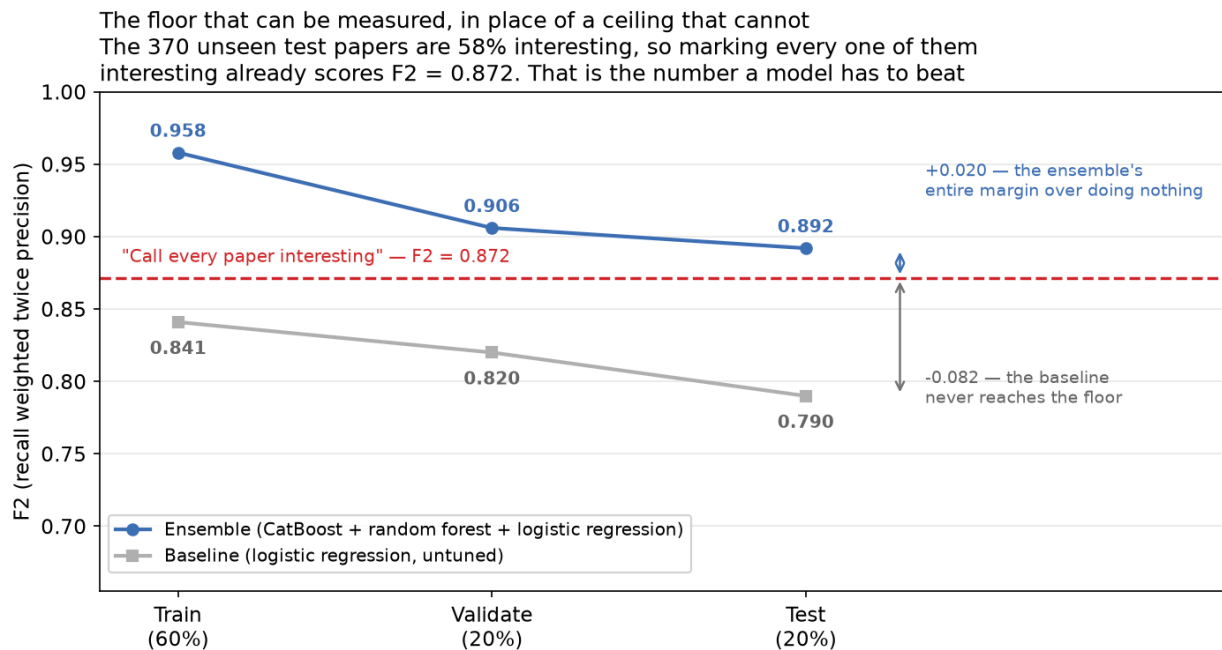


Figure 8. The two models against the do-nothing policy, on the same rows. $F2$ for "call every paper interesting" is fixed by the composition of the pool at 0.872. The untuned baseline (0.790) never reaches it. The ensemble (0.892) clears it by 0.020 — which is the whole of its measured advantage on this metric.

Three things follow, and they are more useful than a guessed ceiling:

- **The baseline's 0.790 $F2$ is worse than doing nothing** on a pool this dense. That is a property of the corpus, not a bug in the model — and it is the clearest statement of why this dataset was the wrong surface for optimising $F2$.
- **The ensemble's real $F2$ margin is +0.020, not the +0.101 often quoted.** The larger figure compares a *tuned* ensemble against an *untuned* baseline; like-for-like the gap is +0.020, and it is not statistically significant.
- **The ranking claim is the one that survives.** The ensemble is a genuinely better *ranker* (+0.022 ROC-AUC, $p=0.005$), which is the property that actually matters, because the analyst reads from the top of a list rather than from everything above a threshold.

None of this transfers to production unchanged. At 1.86% interesting the do-nothing floor collapses from 0.872 to 0.087, so F2 becomes a discriminating metric again — which is precisely why the larger, sparser corpus in Future Work exists.

Model Insights

The model was not the bottleneck. Framing the problem correctly was worth far more than any algorithm choice.

The Ensemble Barely Beats its Component Models

Judged fairly — each model cross-validated at its *own* best decision threshold rather than a shared one — logistic regression scores 0.903 F2, CatBoost 0.902 and random forest 0.898. A spread of 0.005. The ensemble reaches 0.910 on that same surface, so its advantage comes from *combining* the three, not from any one of them being better. (These are cross-validated figures across all folds, which is why they sit above the 0.892 single-slice test score in the table above; they are comparable to each other, not to it.) Relatedly, three different hyperparameter search strategies landed within 0.0024 of each other: at this budget the tuning strategy barely matters either.

Ranking still has a place, thresholds do not travel.

A probability threshold calibrated on a pool of academic papers that is 58% interesting is meaningless on a pool that is 2% interesting. Ranking order still works; a fixed threshold does not. So the operating instruction becomes "read the top 10%", not "read everything above 0.7". This is also why the ensemble's surviving claim is a ranking claim (+0.022 ROC-AUC, $p=0.005$) rather than an F2 claim — see *What these scores have to beat*, above.

Each Use Case has its own level, and the spread is wide.

Fitting per Use Case, as rule 2 of Section 4 requires, gives six separate scores rather than one. The spread between them is 0.090 — wider than any difference between algorithms

found anywhere in this project. Part of that is Use Case quality: missing, ambiguous, conflicting or simply wrong inputs. Part of it is that some Use Cases are intrinsically harder to define.

Use Case	Labels	% interesting	ROC-AUC*	F2 for marking everything interesting
Low-carbon cement	262	65%	0.906	0.90
Carbon capture	295	50%	0.887	0.83
Solar cells for satellites	358	77%	0.846	0.94
Technology prediction	264	60%	0.841	0.88
Named entity recognition	312	72%	0.836	0.93
Soil microbiome	357	26%	0.816	0.64
Total / mean	1,848	58%	0.855	0.87

** Interpreting ROC-AUC: pick one interesting paper and one not-interesting paper at random, how often does the model rank the interesting one higher? 0.5 is a coin flip. 1.0 is perfect. The final column is why ROC-AUC is quoted here rather than F2: F2 for these Use Cases is dominated by how dense each pool is, so it ranks them almost in reverse. Soil microbiome is the sparsest pool and therefore the hardest F2 to score well on and the most informative ROC-AUC — it is also the Use Case where the analyst's definition of interesting was tightest.*

Does an LLM do Better?

Four open-weights language models from 20 billion to 397 billion parameters were tested across four prompt designs, producing 89,937 responses, on the 8 largest external screening projects (62,229 papers, 2.19% interesting). The pass mark was fixed before the run: beat the free zero-label baseline on at least 6 of 8 projects. Every model won exactly 3 of 8. Fail. **The LLM does not replace the ensemble.**

A better Use Case beats a bigger LLM

The entire performance spread across four model families from 20 billion to 397 billion parameters is 0.030 — smaller than the +0.057 that rewriting the Use Case bought on a single 31-billion-parameter model. Same model, same prompt, better Use Case beats a model roughly 13 times larger.

A strong model was shown 30 interesting and 30 not-interesting papers and asked to *write down the screening rule it inferred*. That “induced Use Case” was then handed to each of the system's components in place of the analyst's original.

Who reads the Use Case	Supplied Use Case → induced Use Case	Change
Keyword-matching features	0.733 → 0.798	+0.065
The LLM reader	0.777 → 0.834	+0.057
Similarity-to-Use Case ranking	0.774 → 0.778	+0.004

Use Case quality is the largest lever this project found, and it can be elicited from the analyst's labelling decisions rather than from their prose.

7. Limitations & Future Work

What did not work

The most valuable asset in the project is the list of things that were measured and rejected. In plain terms:

What was tried	Why it did not work
Eleven metadata features (citation velocity, author count, venue quality, language...)	All eleven together were worth almost nothing, and they <i>hurt</i> on unseen Use Cases — they encode which Use Case a paper came from
Estimating technology readiness from the text	53% agreement with a hand-labelled sample. Barely better than guessing
Journal quality scores from OpenAlex	Essentially uncorrelated with the label. It just re-measures citation count
Adding a fourth and fifth model to the ensemble	Three attempts, three rejections. The models agree with each other too much to add anything
Compressing the embedding inside a Use Case	Helped when transferring between Use Cases, hurt in production. A transfer trick, not a production one
Recalibrating the model's probability outputs	Average benefit $\approx$ zero, with noise 50× the average, and <i>negative</i> on one Use Case

What was tried	Why it did not work
Using a prompted LLM instead of the ensemble	Loses on ranking; fails a pre-registered 6-of-8 bar as a cold-start replacement
Giving the LLM reader curated keyword lists	Ranking unchanged, and recall got <i>worse</i> . Keyword lists help keyword matchers and hurt readers
Asking the LLM to check criteria one by one	Worse on 3 of 4 models, the scaffolding makes it demand that <i>every</i> criterion be met
Scoring how well-written a Use Case is, with a single number	Three independent attempts, three failures. There is a structural reason: a defect that is expensive before labelling is cheap after, and vice versa, so no single number can average both regimes

Future Work

This report recommends the following actions.

In Progress

Everything above was measured on the small six-Use-Case dataset the TIRI team built, at an interesting-paper rate roughly thirty times higher than production. To fix that, a larger training set was sought out. Published meta-reviews supply one: **twenty-eight screening projects, 181,199 papers, 1.86% interesting overall**. These carry real, expert labels, with Use Cases written from each review's methods section and without sight of the labels.

	TIRI, the six in-house Use Cases	New training set
Silos	6 Use Cases	28 screening projects

	TIRI, the six in-house Use Cases	New training set
Papers	2,873 harvested · 1,852 labelled · 1,848 modelled	181,199
% interesting	26–77% (pooled 58%)	0.16–78% (pooled 1.86%)
Labelled by	One in-house analyst	Published review authors
Use Cases	Analyst, hand-written	Drafted by a large language model, without sight of labels
Role	Development; the central finding	Realistic interesting rates; the language-model contest

This training set will form the basis of the next training iteration and model build.

Next Steps

- *Find the crossover point where an inferred Use Case stops adding value.* It is worth +0.057 at around 60 labels and roughly nothing once a Use Case has enough labels to train on properly.
- *Measure inter-annotator agreement.* No figure exists for how often two people labelling the same paper against the same Use Case agree — which is why this report can state a measured floor for its scores but not a ceiling. Sampling the *not interesting* papers and re-labelling them blind would close both this and the related gap that a labeller's own miss is indistinguishable from a genuine rejection.
- *Re-run every F2 number at production density.* At 1.86% interesting the do-nothing floor drops from 0.872 to 0.087, so F2 becomes discriminating again and model selection can be redone on a surface where it means something.

- *Test whether the winning ten features stay stable as a Use Case matures.* Given that "interesting" is shown to drift as an analyst learns, it is worth checking whether the feature set that beat the embedding at one point in a Use Case's life is still the best several labelling rounds later, or whether the right features themselves have a lifecycle.
- *Rebuild the Solar cells for satellites pool.* Its labels track publication vintage rather than content, because the pool was seeded from citation-ranked canon. Re-harvesting it without that seed would turn the project's one control failure into a sixth usable Use Case.
- *Investigate why every open-weights language model independently landed on exactly 3 of 8 projects.*
- *Bring in full text and citation graphs, not just abstracts.*
- *Trial the language model as a narrow arbiter rather than a cold scorer.* Rather than asking it to rank the full pool — a competition it loses to the ensemble — test it only on the ambiguous middle of the ensemble's ranked list, where the ensemble itself is least confident.

About the Authors



Sylvain Fossier

Ecology and Genetics | Data Science |
Product Management

linkedin.com/in/sylvain-fossier

github.com/sylvain-gitty



Warren Fauvel

Company Building | Climate Tech |
Product Management

linkedin.com/in/warrenof

github.com/warrenof

With Thanks!

We would like to express our gratitude for the support, critique and encouragement of our coaches, peers and partners in the creation of this project.